

GUÍA DEL DEMOSTRADOR

Lo que necesitas saber

Los conceptos detrás de las seis demostraciones



Para quien lleva la parte práctica
Leer completa antes del ensayo

Contenido

Antes de empezar · los tres errores que casi todos traen	3
Los cinco conceptos, en el orden en que los vas a usar	3
Los dos mecanismos que se ven en pantalla	5
Dónde viven las skills	6
Qué enseña cada demo	7
El ciclo Claude A / Claude B	7
Las herramientas de diagnóstico	7
Las siete cosas que no debes decir	8
Si alguien te pregunta algo que no sabes	8
Lo que ya no tienes que preparar	9

Lo que necesitas saber para narrar las seis demostraciones y para que no te agarren en curva. No es el manual de la audiencia: es más corto y está ordenado por lo que vas a decir en vivo.

Si nunca has escrito una skill, esta guía es suficiente. Léela completa una vez, de principio a fin, antes del ensayo.

Antes de empezar • los tres errores que casi todos traen

Estos tres puntos circulan mal en tutoriales y publicaciones. Si los dices mal en vivo, alguien de la audiencia te va a corregir en el chat, y eso cuesta más que cualquier demo fallida.

1 • El comando sale del nombre del directorio, no del campo `name`.

En skills personales y de proyecto, si el directorio se llama `revisando-cambios`, el comando es `/revisando-cambios`, aunque el campo `name` diga otra cosa, y aunque no haya campo `name`. El campo es una etiqueta del listado y no es obligatorio. Solo en skills que vienen dentro de un plugin el nombre se compone distinto, con el prefijo del plugin.

2 • El tope de la descripción es de 1,536 caracteres.

Y es **combinado** entre `description` y `when_to_use`. Vas a encontrar escrito por todas partes que son 1,024. No lo son.

3 • El cuerpo de la skill se queda en contexto después de invocarla.

Esta es la más importante de las tres y la que cambia cómo se escribe una skill. Cuando una skill se invoca, su cuerpo entra al contexto y **sigue ahí en los turnos siguientes** de esa conversación. No es un préstamo momentáneo que se devuelve.

Consecuencias, y las dos se dicen en la demo 2:

- Cada línea del cuerpo es un costo **recurrente**, no único.
- El cuerpo se escribe como instrucción permanente, no como pasos de un solo uso.

Los cinco conceptos, en el orden en que los vas a usar

1 • Qué es una skill

Un directorio con un archivo `SKILL.md`. Nada más. El archivo tiene dos partes separadas por tres guiones: metadatos arriba, instrucciones abajo.

```
---
description: Qué hace. Úsala cuando el usuario...
---

## Instrucciones

Lo que Claude debe hacer.
```

Esto es todo lo que hace falta para que funcione. El resto — `allowed-tools`, archivos de referencia, scripts — es opcional y se agrega cuando hace falta.

2 · Las dos mitades y sus dos costos · el concepto central

Es el concepto del que dependen los otros cuatro. Se demuestra en la demo 2 y conviene que lo puedas decir de tres formas distintas.

PARTE	CUÁNDO ESTÁ EN CONTEXTO	COSTO
La descripción	Siempre, en todas las conversaciones	Permanente, pero pequeño
El cuerpo	Solo al invocarse, y de ahí en adelante en esa conversación	Nada mientras no se use

La forma corta, para decirla en vivo:

La descripción siempre está en contexto. El cuerpo, no. Ese es todo el truco.

La forma con número, que es la que convence a los escépticos:

Con cincuenta skills instaladas, lo que está ocupando contexto de manera permanente son cincuenta descripciones de dos líneas. Los cincuenta cuerpos están en disco, esperando.

3 · La descripción es lo único que decide

Claude nunca ve el cuerpo de tu skill antes de decidir usarla. Lo único que ve es la descripción. Entonces:

Una skill con cuerpo perfecto y descripción mala no existe en la práctica.

Las cuatro reglas de una descripción que funciona:

Tercera persona. "Revisa", "Analiza", nunca "Te puedo ayudar". La descripción se inyecta en la indicación del sistema, donde Claude lee sobre sí mismo en tercera persona. El cambio de persona rompe el descubrimiento.

Qué hace, más cuándo usarla. Las dos mitades son obligatorias, y la que casi todos omiten es la segunda, que es justo la que decide si se dispara.

El caso principal al inicio. Hay tope de caracteres, y si se recorta, se recorta por el final.

Las palabras que la gente teclea de verdad. No las del manual. El descubrimiento es, en la práctica, coincidencia entre lo que el usuario escribe y lo que dice la descripción. Si la descripción dice "gestión documental" y el usuario escribe "revisa mi README", no se encuentran.

El truco práctico, y es el más útil de todo el webinar: di en voz alta cómo le pedirías la tarea a un compañero, y de esa frase saca las palabras de la descripción.

4 · Grados de libertad

La pregunta no es "*¿cuánto detalle pongo?*" sino "*¿qué tan frágil es esta tarea?*".

La metáfora, que es la que hace que se entienda:

Un robot que tiene que cruzar de un lado a otro. En campo abierto le dices "ve al otro lado" y cualquier ruta sirve. En un puente angosto con precipicios a los lados, le das el paso exacto y le prohíbes improvisar.

GRADO	CUÁNDO	CÓMO SE ESCRIBE
Alta	Varios caminos sirven	Prosa, dirección general
Media	Hay un patrón preferido	Plantilla o script con parámetros
Baja	Se puede romper algo	Comando exacto y prohibición explícita de variarlo

En libertad baja, **la prohibición explícita es parte del diseño**, no un adorno. Sin ella, un modelo capaz va a intentar arreglar el problema que encuentre, y eso es exactamente lo que no quieres en una migración de base de datos.

El error típico: escribir todas las skills con el mismo nivel de detalle. Las de libertad alta quedan asfixiadas y las de baja, peligrosamente vagas.

5 • Revelación progresiva

Una skill puede tener más archivos que el `SKILL.md`, y cada tipo tiene un destino distinto. Esto es lo que se demuestra en la demo 5.

PIEZA	QUÉ LE PASA	EL VERBO CORRECTO
<code>SKILL.md</code>	Se carga al invocarse	—
<code>reference.md</code>	Se carga solo si hace falta	"consulta"
<code>scripts/algo.sh</code>	Se ejecuta , no se lee	"ejecuta"

Las tres reglas donde se equivoca todo el mundo:

Un solo nivel de referencias. El `SKILL.md` apunta al `reference.md` y ahí se acaba. Una cadena de dos niveles funciona a veces y falla otras, que es peor que fallar siempre.

Los scripts se ejecutan. Un script de cien líneas que se *lee* ocupa cien líneas de contexto y no hace nada. Ejecutado, ocupa las quince líneas de su salida y hace el trabajo.

Las rutas usan `${CLAUDE_SKILL_DIR}`. Nunca rutas absolutas. La variable funciona en tu máquina, en la de la audiencia y en un contenedor de integración continua.

Los dos mecanismos que se ven en pantalla

Son los dos detalles técnicos que la audiencia va a preguntar, y los dos aparecen en las demos. Necesitas poder explicarlos sin leer.

El contexto dinámico

Una línea del cuerpo escrita entre acentos invertidos y precedida de un signo de admiración ejecuta un comando **cuando la skill entra**, y pega el resultado en el contexto.

```
## Cambios actuales
```

```
!`git diff HEAD`
```

Por eso en la demo 1 nadie le pega el diff a Claude: la skill lo trae sola.

La trampa, y hay que decirla: si ese comando falla, **se aborta la invocación completa** de la skill. En un repositorio sin commits, `git diff HEAD` falla y la skill entera no entra. Por eso en producción se escribe con una red:

```
!`git diff HEAD || true`
```

El patrón anti-permiso

Es el detalle más valioso de la demo 5. La ruta del script aparece en **dos lugares**, escrita idéntica:

```
allowed-tools: Read Grep Bash(bash ${CLAUDE_SKILL_DIR}/scripts/verificar.sh *)
```

```
bash ${CLAUDE_SKILL_DIR}/scripts/verificar.sh
```

Como coinciden exactamente, el script corre **sin pedir permiso**. Sin este patrón, cada ejecución interrumpe al usuario con una pregunta, y una skill que interrumpe es una skill que nadie usa.

Las dos aclaraciones que evitan el malentendido más frecuente:

- `allowed-tools` **otorga** permiso, **no lo quita**. No es una lista de restricción, y no sirve para limitar lo que una skill puede hacer.
- Se limpia en el mensaje siguiente. No es una puerta que quedó abierta.

Dónde viven las skills

UBICACIÓN	RUTA	ALCANCE
Empresarial	Configuración administrada	Toda la organización
Personal	<code>~/.claude/skills/</code>	Todos tus proyectos
Proyecto	<code>.claude/skills/</code>	Ese repositorio
Plugin	<code><plugin>/skills/</code>	Donde esté activo

En conflicto de nombres: empresarial gana sobre personal, personal sobre proyecto.

El punto que importa para la audiencia, y se dice en el bloque de Erick:

El nivel de proyecto es el que cambia la vida en equipo. Una skill ahí se versiona con el código: quien clona el repositorio hereda su criterio sin que nadie le explique nada. No hay instalación ni aviso.

Qué enseña cada demo

No memorices los pasos: memoriza qué concepto carga cada demo. Si te pierdes en vivo, esto te dice qué tienes que dejar dicho antes de cerrar.

#	DEMO	EL CONCEPTO QUE CARGA	SI SOLO ALCANZAS A DEJAR UNA COSA
1	La skill en tres minutos	Que esto existe y funciona sin comandos	Que nadie le dijo el criterio; estaba en un archivo
2	Anatomía de un SKILL.md	Las dos mitades y sus dos costos	Descripción siempre en contexto, cuerpo no
3	La descripción, A/B	Que la descripción es lo único que decide	Que si no se dispara, el problema no está en el cuerpo
4	El destilador	Cómo construyen la suya mañana	El prompt destilador, leído completo
5	Multiarchivo y script	Revelación progresiva y el patrón anti-permiso	Los scripts se ejecutan, no se leen
6	La skill que no funciona	Diagnóstico, y que la peor falla no la marca ninguna herramienta	Que la descripción mala pasa todas las validaciones

La demo 4 es la que no se recorta. Es lo único del webinar que la audiencia puede usar esa misma tarde. Si el reloj aprieta, se recorta de las demos 5 y 6.

El ciclo Claude A / Claude B

Es el marco de la demo 4 y conviene que lo puedas explicar sin el guion, porque es la pregunta más probable del chat.

A es la sesión donde resolviste el problema a mano y de donde destilas la skill. **B** es una sesión limpia, sin nada de ese contexto, donde la pruebas. Lo que observas en B regresa a A como corrección.

Y la frase que hace que la audiencia se relaje, que es la más importante de la demo 4:

Nadie escribe un SKILL.md bueno de memoria y a la primera.

Por qué hace falta la sesión limpia: en la sesión A, Claude ya tiene en contexto todo lo que le explicaste. Si pruebas ahí, la skill parece funcionar aunque la descripción sea mala, porque el contexto de la conversación está haciendo el trabajo. La prueba solo es válida en limpio.

Las herramientas de diagnóstico

Las usas en la demo 6 y son lo que te salva si algo no carga en el ensayo.

COMANDO	PARA QUÉ
<code>/skills</code>	Qué cargó y de dónde viene cada una

COMANDO	PARA QUÉ
<code>/doctor</code>	La instalación completa, y marca frontmatter roto
<code>/context</code>	Qué está ocupando la ventana ahora mismo
<code>claude --debug</code>	Errores de parseo, cuando una skill simplemente no aparece
<code>claude plugin validate <dir></code>	Recorre un directorio y valida el frontmatter de todas

`claude plugin validate` requiere v2.1.233 o más nueva. Si tu versión es anterior, en la demo 6 di la versión en voz alta y sigue con `--debug`.

Las siete cosas que no debes decir

Errores de precisión que suenan bien y son falsos. Cada uno tiene su versión correcta al lado.

NO DIGAS	DI
"El campo <code>name</code> define el comando"	"El nombre del directorio define el comando"
"El tope de la descripción son 1,024 caracteres"	"Son 1,536, combinados con <code>when_to_use</code> "
"El cuerpo se carga solo mientras se usa"	"El cuerpo se queda en la conversación después de invocarla"
" <code>allowed-tools</code> restringe lo que la skill puede hacer"	" <code>allowed-tools</code> otorga permiso previo; no restringe nada"
"Las skills reemplazan a CLAUDE.md"	"CLAUDE.md es para lo que hace falta siempre; skill, para lo que hace falta a veces"
"Claude lee la skill y decide si aplica"	"Claude solo ve la descripción antes de decidir; el cuerpo lo ve después"
"Puedes anidar referencias todo lo que quieras"	"Un solo nivel. Las cadenas funcionan a veces, que es peor que nunca"

Si alguien te pregunta algo que no sabes

Pasa, y la respuesta buena es siempre la misma:

No lo sé de memoria y prefiero no inventarlo. Lo anoto y va en el correo de seguimiento con la respuesta verificada.

Y luego **anótalo de verdad**, porque Erick va a mandar ese correo.

Lo que no haces: adivinar con tono seguro. En una audiencia de desarrolladores hay siempre alguien que ya lo probó, y una respuesta inventada que alguien corrige en el chat cuesta más credibilidad que diez "no lo sé".

Si la pregunta es conceptual, comercial, o sobre cómo adoptarlo en un equipo, **pásala a Erick**:

Eso lo contesta mejor Erick. Erick, ¿esta es tuya?

| Lo que ya no tienes que preparar

Para que quede claro el alcance, porque la lista de arriba puede parecer mucho:

- Las seis demos están escritas paso a paso, con los comandos exactos.
- El texto hablado está redactado; puedes decirlo tal cual.
- Cada demo trae una tabla de qué hacer si falla, con la respuesta ya escrita.
- Los respaldos son corridas reales ya capturadas: si algo se cae, abres una pestaña y sigues leyendo.
- El ambiente se arma con un script y se verifica con otro.
- Las preguntas probables están respondidas en el banco.

Lo único que no está escrito es tu ensayo. Es también lo único que no tiene sustituto.